

Practical - Week-1

Task-1

1. Develop a C program that demonstrates how a Linux BTL operating system executes a command entered by a user that
 1. Accept a Linux command as input.
 2. Create a child process using fork().
 3. Execute the command in the child process using an appropriate exec() system call.
 4. Allow the parent process to wait for the child using wait().
 5. Display the Process ID (PID) of both parent and child processes.
- 5.

```
GNU nano 7.2 process.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_ARGS 20

int main()
{
    char command[100];
    char *args[MAX_ARGS];
    int count = 0;

    // Accept command from the user
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    // Split the command into arguments
    char *token = strtok(command, " ");

    while (token != NULL && count < MAX_ARGS - 1)
```

```

while (token != NULL && count < MAX_ARGS - 1)
{
    args[count] = token;
    count++;
    token = strtok(NULL, " ");
}

args[count] = NULL;

// Create a child process
pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

// Child process
if (pid == 0)
{
    printf("\nChild Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID : %d\n", getppid());

    // Execute the command
    root@Shamikahasini:~# gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is no
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@Shamikahasini:~# nano process.c
root@Shamikahasini:~# nano process.c
root@Shamikahasini:~# gcc process.c -o process
root@Shamikahasini:~# ls
FIRSST.c          OSSP-2          command_resolver.c  parser          process.c
FIRSST.c.save     OSSP1          fork_demo           parser.c        task1
OSSP              command_resolver  fork_demo.c        process         task1.c
root@Shamikahasini:~# ./process
Enter a Linux command: pwd

Parent Process
Parent PID : 612
Child PID : 616

Child Process
Child PID : 616
Parent PID : 612
/root

Child process has completed.
root@Shamikahasini:~#

```

Observation: The program successfully accepted a Linux command as input and created a child process using `fork()`. The child process executed the entered `ls` command using `exec()`, while the parent process waited for the child process using `wait()`. The Parent PID and Child PID were displayed successfully, and the child process completed execution successfully.

Task-2

2. Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```

root@Shamikahasini:~# pwd
/root
root@Shamikahasini:~# uname -a
Linux Shamikahasini 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2025
x86_64 GNU/Linux
root@Shamikahasini:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:   0-11
Vendor ID:              GenuineIntel
Model name:             13th Gen Intel(R) Core(TM) i5-1335U
CPU family:             6
Model:                 186
Thread(s) per core:     2
Core(s) per socket:     6
Socket(s):              1
Stepping:               3
BogoMIPS:               4992.01
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush
                        e sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopol
                        nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1
                        c movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf
                        fetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase
                        1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsavec
                        1 xsave avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b
                        ear serialize ibt flush_lld arch_capabilities

Vulnerabilities:
Gather data sampling:  Not affected
Ghostwrite:           Not affected
Indirect target selection: Not affected
Itlb multihit:        Not affected
L1tf:                 Not affected
Mds:                  Not affected
Meltdown:             Not affected
Mmio stale data:      Not affected
Old microcode:        Not affected
Reg file data sampling: Vulnerable: No microcode
Retbleed:             Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass:    Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:           Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:           Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS
Srbds:                Not affected
Tsa:                  Not affected
Tsx async abort:      Not affected
Vmscape:              Not affected

root@Shamikahasini:~# lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M 1 disk
sdb   8:16   0 159.4M 1 disk
sdc   8:32   0    2G  0 disk [SWAP]
sdd   8:48   0    1T  0 disk /mnt/wslg/distro

```

Observation: The Linux terminal commands successfully provided information about the system's hardware resources and operating system services. The `uname` command displayed kernel and system information, while `lscpu` showed CPU architecture, 12 CPUs, 6 cores, and 2 threads per core. The `lsblk` command displayed the available storage devices and swap space. The `ps` and `top` commands showed running processes and their resource usage. The OS abstracts CPU, memory, storage, and I/O devices by managing them through system services and providing applications with controlled interfaces instead of requiring direct access to the hardware.